

DJANGO FAST UV

Complete Software Architecture Document

Version 1.0 | June 2026



TABLE OF CONTENTS

1. Executive Summary

2. Market Research & Competitive Analysis

3. Problem Analysis

4. Product Design

5. System Architecture

5.1 Design Principles

5.2 High-Level Architecture

5.3 Package Structure

5.4 Class Architecture

5.5 Data Flow Architecture

5.6 Configuration System

5.7 Plugin System Architecture

6. Implementation Roadmap

- 7. Feature Design**
- 8. Developer Experience Design**
- 9. Testing Strategy**
- 10. CI/CD & PyPI Publishing**
- 11. Security Considerations**
- 12. Scalability & Performance**
- 13. Implementation Order**
- 14. Design Decision Log**
- 15. Risk Analysis**
- 16. Success Metrics**
- 17. Conclusion**



1. EXECUTIVE SUMMARY

Django Fast UV is a next-generation CLI framework that scaffolds production-ready Django projects with modern tooling (UV, Ruff, Pyright), modular generators, and an extensible plugin architecture. Unlike static boilerplates (Cookiecutter Django, Django Edge) or paid SaaS templates, Fast UV is an open-source, composable CLI that lets developers pick exactly what they need -- auth, APIs, Docker, Tailwind -- and

generates a clean, maintainable codebase following Django best practices.

The name "Fast UV" communicates speed (fast project creation) and modern Python tooling (UV, the Rust-based Python package manager that is replacing pip/poetry in 2026).



2. MARKET RESEARCH & COMPETITIVE ANALYSIS

2.1 Current Ecosystem Pain Points

Django developers face consistent friction in project setup:

- Boilerplate fatigue: Cookiecutter Django requires answering 30+ prompts; developers often accept defaults and end up with bloat they don't need.
- Paid templates dominate SaaS scaffolding: AI SaaS Template (\$199), Django SaaS Boilerplate -- good but closed-source and opinionated.
- No modern Python tooling integration: Most templates still use pip/poetry; UV (10-100x faster) is not yet integrated into Django generators.
- Static templates vs. living CLI: Cookiecutter generates once; there's no upgrade path or plugin ecosystem.
- DevOps gap: 71% of Django developers don't use config management tools, indicating a gap for built-in DevOps scaffolding.

2.2 Competitor Analysis Table

Product: Cookiecutter Django

Type: Template

Stars: 22,000+

Pricing: Free

Strengths: Mature, flexible, community-driven

Weaknesses: Static, 30+ prompts, no upgrade path, slow generation

Opportunity: Dynamic CLI with selective feature inclusion

Product: Django Edge

Type: Boilerplate

Stars: ~500

Pricing: Free

Strengths: Modern, Tailwind, minimal

Weaknesses: Limited features, no CLI, no plugin system

Opportunity: Full CLI with plugin architecture

Product: AI SaaS Template

Type: Paid Boilerplate

Stars: N/A

Pricing: \$199

Strengths: SaaS-ready, Stripe, allauth

Weaknesses: Closed source, expensive, opinionated

Opportunity: Open-source alternative with modular billing/auth

Product: Django SaaS Boilerplate

Type: Paid Boilerplate

Stars: N/A

Pricing: \$99-299

Strengths: Multi-tenancy, team management

Weaknesses: Closed source, over-engineered for simple projects

Opportunity: Modular approach: pick only what you need

Product: Wasp (framework)

Type: Full Framework

Stars: 15,000+

Pricing: Free

Strengths: Declarative auth, 10 lines of code

Weaknesses: Not Django; forces new paradigm

Opportunity: Bring Wasp-like DX to Django ecosystem

Product: UV (tool)

Type: Package Manager

Stars: N/A

Pricing: Free

Strengths: 10-100x faster than pip

Weaknesses: No Django integration

Opportunity: Deep UV integration for instant env setup

2.3 Market Gap Identified

There is no open-source, modular CLI tool for Django that:

- 1. Uses modern Python tooling (UV, Ruff, Pyright) by default**
- 2. Allows selective feature inclusion (not all-or-nothing templates)**
- 3. Provides an upgrade path via plugin system**
- 4. Generates clean, documented, production-ready code**
- 5. Is free and community-extensible**



3. PROBLEM ANALYSIS

3.1 Why Django Developers Waste Time

1. Environment Setup (2-4 hours/project): Creating virtualenv, installing Django, configuring settings split, setting up pre-commit, configuring pytest -- repeated every project.

2. Decision Fatigue: Cookiecutter's 30+ prompts force micro-decisions before writing business logic.

3. Integration Hell: Wiring allauth, DRF, Celery, Docker, Tailwind -- each has conflicting configuration patterns.

4. No Standard Project Structure: Every team invents their own apps/, config/, settings/ layout.

5. DevOps Gap: 71% of Django developers don't use config management tools -- they need generated Docker Compose, GitHub Actions, and deployment configs.

3.2 Problem Statement

Django developers spend the first 2-3 days of every project on repetitive scaffolding, configuration, and tooling setup instead of building features. Existing solutions are either too rigid (static templates) or too expensive (paid boilerplates), and none integrate modern Python tooling like UV.

3.3 Solution Statement

Django Fast UV is an open-source CLI framework that generates production-ready Django projects in under 60 seconds. It uses a modular generator architecture where developers select only the features they need, integrates modern Python tooling (UV, Ruff, Pyright) natively, and provides an extensible plugin system for community contributions.

3.4 Unique Selling Proposition

"The only Django CLI that generates exactly what you need, with modern tooling, in under a minute -- and grows with your project."



4. PRODUCT DESIGN

4.1 Vision

To become the standard way Django developers start projects -- as ubiquitous as create-react-app is for React or rails new is for Rails.

4.2 Mission

Eliminate setup friction in Django development through an open-source, extensible CLI framework that embodies Pythonic simplicity and modern tooling excellence.

4.3 Target Audience

- Primary: Solo Django developers and small teams (1-5 devs) starting new projects weekly
- Secondary: Django consultancies spinning up client projects
- Tertiary: Django educators and bootcamps needing consistent project templates

4.4 Ideal User Persona

Alex, Senior Django Developer at a 3-person startup

- Starts 2-3 new projects per month (prototypes, client work, internal tools)
- Uses VS Code with Ruff extension
- Has switched from Poetry to UV for speed
- Frustrated by Cookiecutter's bloat and paid templates' lock-in
- Wants: `django-fastuv create myproject --auth --api --docker --` working project in 30 seconds

4.5 Primary Use Cases

1. MVP Scaffolding: `fastuv create myproject --api --auth --docker --tailwind -- full-stack SaaS starter`

2. Microservice Generator: `fastuv create service --api-only --no-auth --postgres --redis -- API microservice`

3. Package Generator: `fastuv create package my-django-app -- reusable Django package structure`

4. Feature Addition: `fastuv add celery -- adds Celery config to existing Fast UV project`

4.6 Competitive Advantages

UV-Native: First Django tool built around UV -- instant dependency resolution, no lockfile conflicts.

Composable Generators: Pick features a la carte; never get bloat you didn't ask for.

Plugin Ecosystem: Third-party generators via `fastuv install-plugin`.

Upgrade-Aware: `fastuv upgrade` checks for template updates and applies migrations.

Clean Output: Generated code follows Django best practices, fully typed, fully documented.

4.7 Growth Strategy

Phase 1: Core CLI + 5 essential generators (auth, API, Docker, Tailwind, Postgres)

Phase 2: Plugin marketplace + community generators

Phase 3: AI-assisted project customization (`fastuv create --describe "SaaS with team billing"`)

4.8 Open Source Strategy

License: MIT (permissive, corporate-friendly)

Governance: BDFL + core contributors -- eventual Django Software Foundation affiliation

Funding: GitHub Sponsors + optional paid "Pro Templates" (advanced SaaS patterns)

Community: Discord + GitHub Discussions, monthly contributor calls



5. SYSTEM ARCHITECTURE

5.1 Design Principles

Every architectural decision follows these principles:

- 1. Single Responsibility: Each generator does one thing, one way, perfectly.**
- 2. Open/Closed: Core is closed for modification; open for extension via plugins.**
- 3. Dependency Inversion: Generators depend on abstractions (BaseGenerator), not concrete implementations.**
- 4. Composition over Inheritance: Project features composed from generator instances, not subclassed.**
- 5. Explicit over Implicit: No magic -- every file generated is traceable to a generator with clear intent.**

5.2 High-Level Architecture

The system consists of five layers:

Layer 1 - CLI Layer (Click): fastuv create, fastuv add, fastuv plugin, fastuv doctor

Layer 2 - Command Router: Validates args, loads generators, sequences work

Layer 3 - Generator Layer: Core Generators (built-in), Plugin Generators (pip-installed), User Generators (local path)

Layer 4 - Template Engine (Jinja2): Renders .j2 templates with context from generators

Layer 5 - File System Writer: Atomic writes, Git tracking, Conflict detection

5.3 Package Structure

django-fastuv/ Repository root

```
| -- pyproject.toml UV project config, entry points
| -- README.md
| -- LICENSE
| -- CHANGELOG.md
| -- docs/ MkDocs documentation
| | -- architecture.md
| | -- generators.md
| | -- plugins.md
| | -- tutorials/
| -- src/
| | -- fastuv/ Main package
| | | -- __init__.py
| | | -- __main__.py python -m fastuv
| | | -- cli.py Click entry points
| | | -- config.py Global config, settings
| | | -- exceptions.py Custom exceptions
| | | -- constants.py Version, paths, defaults
| | | -- types.py Type definitions (TypedDict, Protocol)
| |
| | -- core/ Core framework (no generators)
| | | -- __init__.py
| | | -- project.py Project context, metadata
| | | -- filesystem.py Atomic file operations
```

```
| | |-- renderer.py Jinja2 template engine wrapper
| | |-- validator.py Input validation
| | |-- sequencer.py Generator dependency resolution
| | |-- reporter.py CLI output, progress, errors
| |
| |-- commands/ CLI command implementations
| | |-- __init__.py
| | |-- create.py fastuv create
| | |-- add.py fastuv add
| | |-- plugin.py fastuv plugin install/list
| | |-- doctor.py fastuv doctor -- diagnostics
| | |-- upgrade.py fastuv upgrade
| |
| |-- generators/ Built-in generators
| | |-- __init__.py
| | |-- base.py BaseGenerator abstract class
| | |-- registry.py Generator discovery & registration
| | |-- django_core.py Base Django project (always runs)
| | |-- auth.py django-allauth / custom auth
| | |-- api.py Django REST Framework / Ninja
| | |-- database.py PostgreSQL / MySQL / SQLite config
| | |-- frontend.py Tailwind / Bootstrap / HTMX
| | |-- docker.py Docker + Compose
| | |-- celery.py Celery + Redis
| | |-- email.py Email backend config
| | |-- testing.py pytest + coverage
| | |-- ci_cd.py GitHub Actions
| | |-- docs.py MkDocs / Sphinx setup
| |
| |-- plugins/ Plugin system
| | |-- __init__.py
| | |-- manager.py Plugin loading, isolation
| | |-- spec.py Plugin interface (Protocol)
| | |-- loader.py Dynamic import from pip packages
| |
| |-- templates/ Jinja2 templates
| |-- base/ Core Django project templates
```

```

| |-- auth/ Auth-related templates
| |-- api/ API templates
| |-- docker/ Dockerfile, compose.yml
| |-- ... One directory per generator
|
|-- tests/ Test suite
| |-- unit/ Unit tests (pytest)
| |-- integration/ End-to-end CLI tests
| |-- fixtures/ Sample projects for testing
|
|-- scripts/ Development scripts
| |-- lint.sh Ruff check + format
| |-- test.sh pytest with coverage
| |-- release.sh Version bump + tag
|
|-- .github/
|-- workflows/
|-- ci.yml Lint, type-check, test
|-- release.yml PyPI publish on tag
|-- docs.yml Deploy docs to GitHub Pages

```

5.4 Class Architecture

Core Classes:

ProjectContext

Attributes: name (str), root_path (Path), options (dict), selected_generators (list), metadata (dict)

Methods: validate(), to_dict()

BaseGenerator (Abstract)

Attributes: name (str), description (str), priority (int), dependencies (list), conflicts (list)

Methods: render(context), validate(context), generate(context)*, post_generate(context)*

GeneratorRegistry

Attributes: generators (dict)

Methods: register(generator), resolve(names), validate_combination(names), get_dependencies(names)

TemplateRenderer

Attributes: `jinja_env` (Environment)

Methods: `render(template_path, context)`, `render_to_file(template_path, dest_path, context)`, `register_filter(name, func)`

FileSystemWriter

Methods: `write(path, content, mode)`, `mkdir(path)`, `copy(src, dest)`, `exists(path)`, `git_track(path)`

Sequencer

Methods: `resolve_order(generators)`, `detect_cycles(graph)`, `parallel_safe(generators)`

PluginManager

Attributes: `plugins` (list)

Methods: `discover()`, `load(name)`, `install(package)`, `uninstall(package)`

PluginSpec (Protocol)

Attributes: `name` (str), `version` (str)

Methods: `generators()`, `initialize(config)`

CLICommand

Attributes: `name` (str), `params` (dict)

Methods: `execute(**kwargs)`, `validate_args()`

Generator Inheritance Hierarchy:

BaseGenerator (Abstract)

```
|-- DjangoCoreGenerator (creates settings/ split, manage.py, base requirements)
|-- AuthGenerator (django-allauth integration, custom user model, login/logout
templates)
|-- APIGenerator (DRF or Ninja, serializers, viewsets, OpenAPI schema)
|-- DatabaseGenerator (PostgreSQL config, MySQL config, SQLite default)
|-- DockerGenerator (Dockerfile, docker-compose.yml, entrypoint scripts)
|-- FrontendGenerator (Tailwind config, Bootstrap config, HTMX setup)
|-- CeleryGenerator
|-- EmailGenerator
|-- TestingGenerator
|-- CICDGenerator
```

5.5 Data Flow Architecture

User CLI Input -> Parser (Click) -> Validator (schema) -> Generator Pipeline (sequenced) -> Renderer (Jinja2) -> Writer (atomic) -> Generated Project

5.6 Configuration System

Fast UV uses a layered configuration approach:

1. Built-in defaults (fastuv/config.py)

2. User global config (~/.config/fastuv/config.toml)

3. Project local config (./.fastuv/config.toml)

4. CLI flags (highest priority)

Example global config:

```
[defaults]
```

```
python_version = "3.12"
```

```
package_manager = "uv"
```

```
formatter = "ruff"
```

```
type_checker = "pyright"
```

```
[plugins]
```

```
auto_discover = true
```

```
    index_url = "https://pypi.org/simple"
```

```
[output]
```

```
color = true
```

```
verbose = false
```

```
git_init = true
```

5.7 Plugin System Architecture

Plugins are Python packages following the naming convention fastuv-* (e.g., fastuv-stripe, fastuv-graphql).

Plugin interface (Protocol):

```
class PluginSpec(Protocol):
```

name: str

version: str

```
def generators(self) -> list[type[BaseGenerator]]: ...
```

```
def initialize(self, config: dict) -> None: ...
```

Plugin discovery process:

1. Scan `sys.path` for modules matching `fastuv_*`
2. Import and validate against `PluginSpec`
3. Register generators into `GeneratorRegistry`
4. CLI dynamically adds commands based on loaded plugins



6. IMPLEMENTATION ROADMAP

Milestone 1: Foundation (Weeks 1-2)

Goal: Core CLI infrastructure working

Deliverable Complexity Dependencies

```
BaseGenerator abstract class Medium None
```

```
ProjectContext dataclass Low None
```

TemplateRenderer (Jinja2) Low Jinja2

FileSystemWriter (atomic ops) Medium None

GeneratorRegistry Medium BaseGenerator

Sequencer (dependency res) High GeneratorRegistry

CLI entry point with Click Low Click

create command (bare Django) Medium All above

Success Metrics: `fastuv create testproject` generates a runnable Django project.

Milestone 2: Essential Generators (Weeks 3-4)

Goal: Most common features implemented

Deliverable Complexity Dependencies

DjangoCoreGenerator High Foundation

DatabaseGenerator Medium DjangoCore

TestingGenerator Medium DjangoCore

AuthGenerator High DjangoCore, Database

DockerGenerator Medium DjangoCore

CICDGenerator Low None

Success Metrics: `fastuv create myproject --auth --docker --postgres` generates a full project with working auth and Docker.

Milestone 3: Frontend & API (Weeks 5-6)

Goal: Full-stack scaffolding

Deliverable Complexity Dependencies

APIGenerator High DjangoCore, Auth

FrontendGenerator Medium DjangoCore

EmailGenerator Low DjangoCore

CeleryGenerator Medium DjangoCore, Docker

Success Metrics: `fastuv create saas --auth --api --docker --tailwind --celery` generates a SaaS-ready project.

Milestone 4: Plugin System (Weeks 7-8)

Goal: Extensible architecture

Deliverable Complexity Dependencies

PluginManager High Foundation

PluginSpec Protocol Medium PluginManager

Dynamic command registration High PluginManager, CLI

fastuv plugin commands Medium PluginManager

Example plugin (fastuv-hello) Low Plugin system

Success Metrics: Third-party plugin can add a new generator and CLI command.

Milestone 5: Polish & Release (Weeks 9-10)

Goal: Production-ready v1.0

Deliverable Complexity Dependencies

doctor command Medium All

upgrade command High All

add command High All

Comprehensive test suite High All

Documentation site (MkDocs) Medium None

PyPI release workflow Low None

Success Metrics: 100% test coverage on core, published to PyPI, 50+ GitHub stars in first week.



7. FEATURE DESIGN

7.1 Django Core Generator

Problem: Every Django project needs settings split, environment variables, and a standard directory layout. Developers recreate this every time.

Solution generates:

myproject/

```
| -- config/
```

```
| | -- __init__.py
```

```
| | -- settings/
```

```
| | | -- __init__.py
```

```
| | | -- base.py
```

```

| | |-- local.py
| | |-- test.py
| | |-- production.py
| |-- urls.py
| |-- wsgi.py
| |-- asgi.py
|-- apps/ User apps go here
|-- requirements/
| |-- base.txt
| |-- local.txt
| |-- production.txt
|-- .env.example
|-- manage.py
|-- pyproject.toml UV config, Ruff, Pyright

```

Design Decision: Settings split into directory with inheritance rather than settings_local.py pattern. This is the Django best practice endorsed by Two Scoops of Django and prevents accidental production settings exposure.

7.2 Auth Generator

Problem: django-allauth configuration is complex (10+ settings, URL patterns, adapter classes). Custom user models require migration discipline.

Solution:

- Generates custom User model inheriting from AbstractUser in first migration
- Configures django-allauth with email-as-username, social providers ready
- Creates login/logout/password-reset templates with selected frontend framework
- Sets up ACCOUNT_EMAIL_VERIFICATION = "mandatory" for production safety

Design Decision: Always create custom User model even if not immediately needed. Adding later requires painful migration. This prevents the #1 Django production mistake.

7.3 API Generator

Problem: DRF requires serializers, viewsets, routers, permission classes, pagination, filtering -- 50+ lines of boilerplate per resource.

Solution:

- Option A: Django REST Framework (mature, batteries-included)
- Option B: Django Ninja (type-safe, OpenAPI auto-generated, faster)
- Generates api/ app with base classes
- Includes UserSerializer, UserViewSet wired to auth
- Auto-generates OpenAPI schema at /api/schema/
- Configures DEFAULT_PERMISSION_CLASSES to require auth

Design Decision: Support both DRF and Ninja. DRF for teams needing ecosystem maturity; Ninja for teams wanting type safety and performance. Default to Ninja for new projects (2026 best practice).

7.4 Docker Generator

Problem: Writing Dockerfiles for Django is error-prone (multi-stage builds, static files, media volumes, entrypoints).

Solution:

- Multi-stage Dockerfile (builder + runtime) for minimal image size
- docker-compose.yml with PostgreSQL, Redis (if Celery), and web service
- Entrypoint script handling migrations and static collection
- .dockerignore optimized for Python projects
- Health checks configured

Design Decision: Use UV in Docker (COPY pyproject.toml uv.lock /app/ then uv sync) for fastest, most reproducible builds. This is 10x faster than pip in container builds.

7.5 Frontend Generator

Problem: Integrating Tailwind with Django requires django-tailwind, node, npm, and watcher configuration.

Solution:

- Tailwind: django-tailwind setup with tailwind init, base template, component classes
- Bootstrap: django-bootstrap-v5, crispy forms, base template
- HTMX: htmx CDN + django-htmx, no build step required

Design Decision: HTMX as default for server-rendered apps (Django's sweet spot), Tailwind for modern SaaS UIs, Bootstrap for rapid admin-style interfaces.



8. DEVELOPER EXPERIENCE DESIGN

8.1 Primary CLI Workflow (Interactive Mode)

```
$ fastuv create myproject
? Python version: 3.12 (default)
? Database: [PostgreSQL] / SQLite / MySQL
? Authentication: [django-allauth] / None / Custom
? API: [Django Ninja] / DRF / None
? Frontend: [Tailwind] / Bootstrap / HTMX / None
? Docker: [Yes] / No
? Celery: [Yes] / No
? Email: [Console] / SendGrid / Mailgun / AWS SES
? CI/CD: [GitHub Actions] / None
```

Generating project...

[OK] Created directory structure

[OK] Rendered 24 templates

[OK] Initialized git repository

[OK] Installed dependencies with UV (2.3s)

[OK] Ran initial migrations

[OK] Created superuser (optional)

Project 'myproject' is ready!

cd myproject

docker compose up If Docker selected

uv run manage.py runserver Or local development

8.2 Non-Interactive Mode (CI/Scripts)

```
$ fastuv create myproject \  
--python 3.12 \  
--database postgres \  
--auth allauth \  
--api ninja \  
--frontend tailwind \  
--docker \  
--celery \  
--email sendgrid \  
--ci github-actions \  
--no-input
```

8.3 Incremental Addition

```
$ cd myproject  
$ fastuv add celery  
[OK] Generated celery.py config  
[OK] Added redis to docker-compose.yml  
[OK] Updated requirements  
[OK] Created tasks/ directory with example task  
  
$ fastuv add stripe  
[OK] Installed fastuv-stripe plugin  
[OK] Generated checkout views  
[OK] Added webhook handlers  
[OK] Created Stripe customer model
```

8.4 Diagnostics

```
$ fastuv doctor  
[OK] Python 3.12.4 found  
[OK] UV 0.4.1 found
```

[OK] Docker 27.0.3 found

[OK] Git 2.45.1 found

[!] PostgreSQL client not found (optional)

[OK] All checks passed!



9. TESTING STRATEGY

9.1 Test Pyramid

Level Framework Coverage Target Examples

Unit pytest 95% Generator logic, template rendering, validation

Integration pytest + Click runner 80% CLI commands, plugin loading

E2E pytest + tmp_path 70% Full project generation, manage.py check passes

Compatibility tox / nox N/A Python 3.12, 3.13; Django 4.2, 5.0, 5.1

9.2 Key Test Patterns

Example: Testing a generator

```
def test_auth_generator_creates_custom_user(tmp_path: Path):
    context = ProjectContext(name="test", root_path=tmp_path, options={})
    generator = AuthGenerator()
    generator.generate(context)
    user_model = tmp_path / "apps" / "users" / "models.py"
    assert user_model.exists()
```

```
    assert "class User(AbstractUser)" in user_model.read_text()
    assert "email = models.EmailField(unique=True)" in user_model.read_text()
```

Example: Testing CLI

```
def test_create_command_runs_successfully(cli_runner, tmp_path):
    result = cli_runner.invoke(
```

```
create,  
["myproject", "--no-auth", "--no-input", "--path", str(tmp_path)]  
)  
assert result.exit_code == 0  
assert (tmp_path / "myproject" / "manage.py").exists()
```

9.3 E2E Test Strategy

For every combination of core generators ($2^5 = 32$ combinations), generate a project and verify:

1. **python manage.py check passes**
2. **python manage.py migrate succeeds**
3. **python manage.py test runs (even if 0 tests)**
4. **docker compose build succeeds (if Docker selected)**

Run nightly, not on every PR (too slow).



10. CI/CD & PYPI PUBLISHING

10.1 GitHub Actions Workflow (CI)

Name: CI

Trigger: push, pull_request

Jobs:

test:

Strategy: matrix

python: ["3.12", "3.13"]

django: ["4.2", "5.0", "5.1"]

Steps:

- Checkout code
- Setup UV (astral-sh/setup-uv@v3, version 0.4.x)
- Run: uv sync --all-extras --dev
- Run: uv run pytest --cov=fastuv --cov-report=xml
- Run: uv run ruff check .
- Run: uv run pyright src/
- Upload coverage to Codecov

e2e:

```
Condition: github.ref == 'refs/heads/main'
```

Dependencies: test job must pass

Steps:

- Checkout code
- Setup UV
- Run: uv run pytest tests/integration/ -v

10.2 Release Workflow

Name: Release

Trigger: push tags matching v*

Jobs:

publish:

Permissions: id-token: write (for trusted publishing)

Steps:

- Checkout code
- Setup UV
- Run: uv build
- Run: uv publish --trusted-publishing

10.3 PyPI Trusted Publishing

Uses PyPI's trusted publisher model (no API tokens in GitHub secrets):

1. GitHub Actions workflow has id-token: write permission

2. PyPI project configured with GitHub as trusted publisher

3. uv publish --trusted-publishing uses OIDC token for authentication

Design Decision: UV handles both building and publishing. No need for build, twine, or hatch separately. This reduces tool surface area and potential supply chain attacks.



11. SECURITY CONSIDERATIONS

Concern Mitigation

Template injection Jinja2 sandboxed environment; no eval or exec in templates

Dependency confusion Pin all dependencies in uv.lock; verify hashes

Secret leakage .env.example never contains real secrets; generated .env in .gitignore

Supply chain UV lockfile with SHA-256 hashes; dependabot/renovate for updates

Plugin isolation Plugins run in same process (Python limitation); code review required for official plugin list

Generated code security SECURE_SSL_REDIRECT, SECURE_HSTS_SECONDS, X_FRAME_OPTIONS set in production settings



12. SCALABILITY & PERFORMANCE

Metric Target Strategy

Project generation time < 5 seconds UV parallel dependency resolution; template caching

CLI startup time < 100ms Lazy imports; plugin discovery cached

Memory usage < 50MB Streaming template rendering; no full project in memory

Concurrent generators 4 parallel ThreadPoolExecutor for I/O-bound template rendering

Supported project size 500+ files Atomic file writes with rollback on failure



13. IMPLEMENTATION ORDER

Phase 1: Core Infrastructure

1. fastuv/types.py Type definitions, TypedDicts, Protocols

2. fastuv/exceptions.py Custom exceptions (FastUVErrors, GeneratorError, etc.)

3. fastuv/constants.py Version, default paths, file patterns

4. fastuv/config.py Configuration loading (TOML, env vars, defaults)

5. fastuv/core/filesystem.py Atomic file operations, rollback support

6. fastuv/core/renderer.py Jinja2 environment, custom filters

7. fastuv/core/project.py ProjectContext dataclass, validation

8. fastuv/core/sequencer.py Topological sort for generator ordering

9. fastuv/core/reporter.py Rich console output, progress bars

Phase 2: Generator Framework

10. fastuv/generators/base.py BaseGenerator ABC with lifecycle hooks

11. fastuv/generators/registry.py Discovery, registration, conflict detection

12. fastuv/generators/django_core.py Base Django project (always included)

Phase 3: Built-in Generators

13. fastuv/generators/database.py PostgreSQL, MySQL, SQLite configs

14. fastuv/generators/testing.py pytest, coverage, factory-boy

15. fastuv/generators/auth.py django-allauth, custom User model

16. fastuv/generators/api.py Django Ninja / DRF

17. fastuv/generators/frontend.py Tailwind, Bootstrap, HTMX

18. fastuv/generators/docker.py Multi-stage Dockerfile, compose

19. fastuv/generators/email.py Email backends

20. fastuv/generators/celery.py Celery + Redis

21. fastuv/generators/ci_cd.py GitHub Actions workflows

Phase 4: Plugin System

22. fastuv/plugins/spec.py PluginSpec Protocol

23. fastuv/plugins/manager.py Discovery, loading, lifecycle

24. fastuv/plugins/loader.py Dynamic import, validation

Phase 5: CLI Commands

- 25. fastuv/commands/create.py fastuv create**
- 26. fastuv/commands/add.py fastuv add**
- 27. fastuv/commands/plugin.py fastuv plugin**
- 28. fastuv/commands/doctor.py fastuv doctor**
- 29. fastuv/commands/upgrade.py fastuv upgrade**
- 30. fastuv/cli.py Click entry point, command registration**

Phase 6: Templates

- 31. fastuv/templates/base/ Core Django project templates**
- 32. fastuv/templates/auth/ Auth templates**
- 33. fastuv/templates/api/ API templates**
- 34. fastuv/templates/docker/ Docker templates**
- 35. fastuv/templates/frontend/ Frontend templates**
- 36. fastuv/templates/ci_cd/ GitHub Actions templates**

Phase 7: Tests & Documentation

37. tests/unit/ Unit tests for each module

38. tests/integration/ CLI integration tests

39. tests/fixtures/ Expected output fixtures

40. docs/ MkDocs documentation site

Phase 8: Packaging & Release

41. pyproject.toml UV project configuration

42. .github/workflows/ci.yml Continuous integration

43. .github/workflows/release.yml PyPI publishing

44. scripts/ Development automation scripts



14. DESIGN DECISION LOG

Decision Rationale Trade-offs

1 UV over pip/poetry 10-100x faster; lockfile with hashes Less mature ecosystem; learning curve

2 Click over Typer De-facto standard; Typer adds complexity More boilerplate than Typer

3 Jinja2 over string tpl Mature, sandboxed, extensible Slightly slower than f-strings

4 Settings dir over file Two Scoops best practice More complex for beginners

5 Django Ninja default Type safety, auto OpenAPI, performance Smaller ecosystem than DRF

6 Plugin naming fastuv-* pytest/flake8 convention Potential naming collisions

7 MIT License Maximum adoption; corporate-friendly No copyleft protection

8 No ORM/model generation Too opinionated; Django commands suffice Users may request later

9 Atomic file writes Rollback on failure Slightly more disk I/O

10 Topological ordering Correct dependency resolution More complex than simple list



15. RISK ANALYSIS

Risk Probability Impact Mitigation

UV adoption stalls Medium High Support pip/poetry as fallback in config

Django Ninja loses momentum Low Medium Keep DRF as alternative; abstract API layer

Plugin ecosystem doesn't grow Medium High Seed with 5 official plugins; GitHub Sponsors

Scope creep (becomes framework) High High Strict "generator only" principle

Maintenance burnout Medium High BDFL + 3 core contributors minimum

Cookiecutter adds similar features Medium Medium Differentiate via live CLI, plugin system, UV



16. SUCCESS METRICS

Metric Target (3 months) Target (12 months)

PyPI downloads/month 1,000 10,000

GitHub stars 200 1,500

Active contributors 3 15

Community plugins 0 10

Issues resolved within 7 days 80% 90%

Test coverage 90% 95%

Average generation time < 5s < 3s



17. CONCLUSION

Django Fast UV fills a critical gap in the Django ecosystem: a modern, open-source, extensible CLI framework that eliminates setup friction while embracing 2026 Python tooling standards. The architecture prioritizes:

- Modularity: Composable generators that do one thing well
- Extensibility: Plugin system for community innovation
- Performance: UV-native for sub-5-second project creation
- Quality: Production-ready output following Django best practices
- Developer Experience: Intuitive CLI with both interactive and scripted modes

The implementation plan is staged across 10 weeks, with each milestone delivering measurable value. The modular architecture allows parallel development -- contributors can work on individual generators without touching core infrastructure.

This completes the software architecture document for Django Fast UV.



Document Version: 1.0

Generated: June 2026

Author: Senior Python Architect & Open-Source Maintainer

